

Министерство Образования Республики Беларусь
УО Брестский Государственный Технический Университет
Кафедра ИИТ

Отчет по лабораторной работе № 4
По дисциплине: «Современный платформы программирования»
Специальность: «Программная инженерия»

Выполнил:
О.А. Козюта
Студент группы ПО-13
Проверил:
А.Д. Кулик
Преп.-стажёр кафедры ИИТ

Брест 2026

Цель работы: научиться работать с Github API, приобрести практические навыки написания программ для работы с REST API или GraphQL API

Общее задание: используя Github API, реализовать предложенное задание на языке Python. Выполнить визуализацию результатов, с использованием графика или отчета. Можно использовать как REST API (рекомендуется), так и GraphQL

8. Анализ активности пользователя на GitHub

Условие:

Напишите Python-скрипт, который:

1. Запрашивает у пользователя имя пользователя GitHub.
2. Получает список публичных репозиториях этого пользователя через GitHub API.
3. Для каждого репозитория получает список последних 10 коммитов и извлекает:
 - Дату коммита
 - Сообщение коммита
 - Автора (если отличается от владельца репозитория)
4. Подсчитывает общее количество коммитов во всех репозиториях.
5. Выводит самые активные месяцы (например, по количеству коммитов).
6. Строит график активности пользователя по месяцам с помощью matplotlib или seaborn.
7. Сохраняет результаты в JSON-файл github_activity.json

```
Введите имя пользователя GitHub: octocat
Общее количество коммитов: 567
Самый активный месяц: Октябрь (80 коммитов)
ТОП-3 репозитория по количеству коммитов:
1. my_project (120 коммитов)
2. cool_repo (95 коммитов)
3. python_scripts (75 коммитов)
Данные сохранены в github_activity.json
График активности построен (activity_chart.png)
```

```
#!/usr/bin/env python3
```

```
"""
```

GitHub User Activity Analyzer

This script fetches public repositories and commits of a GitHub user, analyzes commit activity, and generates a JSON report and a chart.

```
"""
```

```
import sys
import json
from collections import defaultdict
from datetime import datetime
from typing import Dict, List, Optional, Tuple, Any

import requests
```

```
import matplotlib.pyplot as plt
from requests.exceptions import RequestException
```

```
GITHUB_API_BASE = "https://api.github.com"
USER_AGENT = "GitHub-Activity-Analyzer/1.0"
```

```
def get_user_repos(username: str) -> List[Dict[str, Any]]:
    """Fetch all public repositories of a GitHub user."""
    repos = []
    page = 1
    per_page = 100

    while True:
        url = f"{GITHUB_API_BASE}/users/{username}/repos"
        params = {"page": page, "per_page": per_page, "type": "public"}
        headers = {"Accept": "application/vnd.github.v3+json", "User-Agent": USER_AGENT}

        try:
            response = requests.get(url, headers=headers, params=params, timeout=10)
            response.raise_for_status()
            data = response.json()
            if not data:
                break
            repos.extend(data)
            page += 1
        except RequestException as e:
            print(f"Error fetching repositories: {e}", file=sys.stderr)
            sys.exit(1)

    return repos
```

```
def get_all_commits(repo_full_name: str) -> List[Dict[str, Any]]:
    """
    Fetch all commits of a repository (with pagination).
    Returns list of commit objects as returned by GitHub API.
    """
    commits = []
    page = 1
    per_page = 100

    while True:
        url = f"{GITHUB_API_BASE}/repos/{repo_full_name}/commits"
        params = {"page": page, "per_page": per_page}
        headers = {"Accept": "application/vnd.github.v3+json", "User-Agent": USER_AGENT}

        try:
            response = requests.get(url, headers=headers, params=params, timeout=10)
```

```

        response.raise_for_status()
        data = response.json()
        if not data:
            break
        commits.extend(data)
        page += 1
    except RequestException as e:
        print(f"Error fetching commits for {repo_full_name}: {e}", file=sys.stderr)
        break

return commits

```

```

def extract_commit_info(
    commit_data: Dict[str, Any], repo_name: str, owner_login: str
) -> Dict[str, Any]:
    """Extract date, message, and author (if different from owner)."""
    commit = commit_data.get("commit", {})
    author_info = commit.get("author", {})
    date_str = author_info.get("date")
    message = commit.get("message", "").split("\n")[0] # first line only
    author_login = None
    if commit_data.get("author") and commit_data["author"].get("login"):
        author_login = commit_data["author"]["login"]
    else:
        author_login = commit_data.get("committer", {}).get("login")

    author = None
    if author_login and author_login != owner_login:
        author = author_login

    return {
        "repo": repo_name,
        "date": date_str,
        "message": message,
        "author": author,
    }

```

```

def analyze_activity(commits_details: List[Dict[str, Any]]) -> Tuple[int, Dict[str, int], List[Tuple[str, int]]]:
    """
    Compute total commits, monthly activity, and top repositories.
    Returns:
        total_commits,
        month_commits (dict month_name -> count),
        repo_commits (list of (repo_name, count) sorted descending)
    """
    month_counts = defaultdict(int)
    repo_counts = defaultdict(int)

```

```

for item in commits_details:
    if item["date"]:
        try:
            dt = datetime.fromisoformat(item["date"].replace("Z", "+00:00"))
            month_name = dt.strftime("%B") # e.g., "October"
            month_counts[month_name] += 1
        except ValueError:
            pass
    repo_counts[item["repo"]] += 1

total = len(commits_details)
# sort months by count (descending)
sorted_months = dict(sorted(month_counts.items(), key=lambda x: x[1], reverse=True))
# top 3 repos
top_repos = sorted(repo_counts.items(), key=lambda x: x[1], reverse=True)[:3]

return total, sorted_months, top_repos

```

```

def plot_activity(month_counts: Dict[str, int], output_file: str = "activity_chart.png") -> None:
    """Generate a bar chart of commits per month."""
    if not month_counts:
        print("No commit data to plot.", file=sys.stderr)
        return

```

```

    months = list(month_counts.keys())
    counts = list(month_counts.values())

    plt.figure(figsize=(10, 6))
    plt.bar(months, counts, color="skyblue")
    plt.xlabel("Month")
    plt.ylabel("Number of commits")
    plt.title("Commit Activity by Month")
    plt.xticks(rotation=45, ha="right")
    plt.tight_layout()
    plt.savefig(output_file)
    print(f"Chart saved as {output_file}")

```

```

def save_json(data: Dict[str, Any], output_file: str = "github_activity.json") -> None:
    """Save data to JSON file."""
    with open(output_file, "w", encoding="utf-8") as f:
        json.dump(data, f, indent=2, ensure_ascii=False)
    print(f"Data saved to {output_file}")

```

```

def main() -> None:
    """Main entry point."""
    username = input("Введите имя пользователя GitHub: ").strip()

```

```

if not username:
    print("Username cannot be empty.", file=sys.stderr)
    sys.exit(1)

print(f"Fetching repositories for {username}...")
repos = get_user_repos(username)
if not repos:
    print(f"No public repositories found for user {username}.", file=sys.stderr)
    sys.exit(1)

print(f"Found {len(repos)} repositories. Fetching commits (this may take a while)...")

all_commits_details = []
repo_commit_counts = {}

for repo in repos:
    repo_name = repo["name"]
    full_name = repo["full_name"]
    owner_login = repo["owner"]["login"]
    print(f" Processing {full_name}...")
    commits = get_all_commits(full_name)
    if not commits:
        continue
    repo_commit_counts[repo_name] = len(commits)

    # For details we only need last 10 commits (per requirement)
    last_10 = commits[:10]
    for commit in last_10:
        info = extract_commit_info(commit, repo_name, owner_login)
        all_commits_details.append(info)

if not all_commits_details:
    print("No commits found.", file=sys.stderr)
    sys.exit(1)

total_commits, month_counts, top_repos = analyze_activity(all_commits_details)

# Prepare output data
output_data = {
    "username": username,
    "total_commits": total_commits,
    "top_repos": [{"name": name, "commits": count} for name, count in top_repos],
    "monthly_activity": month_counts,
    "commits_details": all_commits_details, # last 10 commits per repo
}

# Display results
print(f"\nОбщее количество коммитов: {total_commits}")
if month_counts:
    top_month = next(iter(month_counts.items()))

```

```
print(f"Самый активный месяц: {top_month[0]} ({top_month[1]} коммитов)")
print("ТОП-3 репозитория по количеству коммитов:")
for i, (name, count) in enumerate(top_repos, 1):
    print(f" {i}. {name} ({count} коммитов)")
```

```
save_json(output_data)
plot_activity(month_counts)
```

```
if __name__ == "__main__":
    main()
```

Вывод: были приобретены навыки применения паттернов проектирования при решении практических задач с использованием языка Python.